

Langkah bicara / persiapan — tidak perlu layar tertentu

► TAMPILKAN / LAKUKAN

- Buka aplikasi IOMS. Siapkan 3 akun: Admin (Rita), Sales (Beni), Warehouse (Wawan).
- Tips: buka 3 jendela / incognito berbeda supaya cepat berpindah peran tanpa logout-login terus.
- Pastikan data demo siap: ada Purchase Order berstatus **Ordered** (PO baru berstatus Draft dan hanya Admin yang bisa men-*submit*-nya) dan Sales Order yang bisa dibuat/disetujui.

KATAKAN (baca naskah ini)

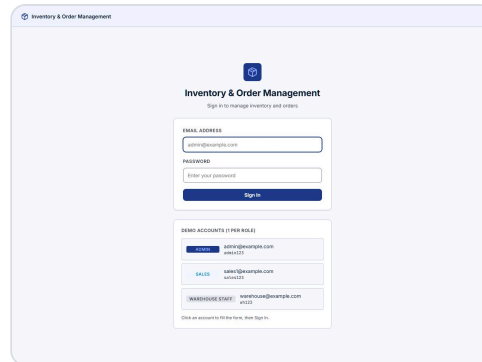
"Sebelum demo, perkenalan singkat: aplikasi ini mengelola stok dan pesanan untuk perusahaan dengan beberapa gudang. Ada tiga pengguna — Admin, Sales, dan Warehouse — dan saya akan mendemokan perjalanan sebuah pesanan dari dibuat sampai barangnya keluar, berpindah dari satu peran ke peran berikutnya."

Kenapa 3 role dipisah — ucapkan di momen ini

Ini Segregation of Duties — supaya tidak ada satu orang yang menguasai seluruh proses (membuat sekaligus menyetujui pesannya sendiri). Kontrol anti-kecurangan klasik.

Peta dokumen ini

Halaman demo (langkah 1-11) diikuti lampiran: **A** Sesi, Redis & bcrypt · **B** Apa yang di-cache Memcached · **C/D/E** Checklist "how-to" per role (Admin, Sales, Warehouse) · **F** Koreksi terhadap versi sebelumnya.



Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

- Tampilkan halaman login. Masukkan email & password, klik Masuk.

🎤 KATAKAN (baca naskah ini)

"Pertama saya login. Saya ketik email dan password lalu Masuk."

💡 Selip alasan — Sesi & Redis — ucapkan di momen ini

"Saat login berhasil, identitas saya disimpan sebagai sesi. Sesi ini saya simpan di **Redis** — bukan di database utama — karena sesi butuh masa berlaku otomatis dan harus cepat dibaca di setiap halaman. Kalau Redis kebetulan mati, aplikasi otomatis beralih ke penyimpanan file, jadi login tetap jalan." • Keamanan: password dicek dengan **bcrypt**, tidak pernah disimpan polos.

Maksud kedua kalimat di atas (terverifikasi di kode)

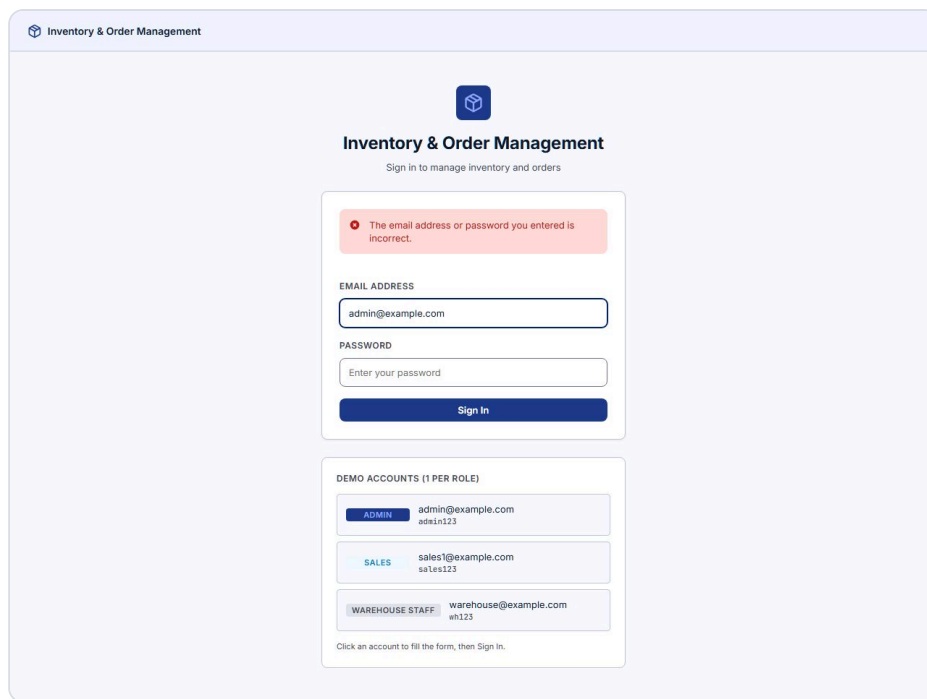
1. "Redis mati → beralih ke file, login tetap jalan"

- **Sesi** = data "siapa yang sedang login" (`user_id` , `role` , `last_activity`) di sisi server; browser hanya memegang cookie berisi ID acak. Normalnya disimpan di **Redis** (cepat, kedaluwarsa otomatis 3600 dtk, tidak membebani MySQL). Hanya sesi di Redis — bukan data bisnis/stok.
- Tiap request, `SessionManager` mengecek koneksi ke Redis (timeout 0,5 dtk). Gagal → tulis peringatan di log dan pakai **sesi berbasis file** (`session.save_handler=files`). Tanpa ini, Redis mati membuat semua request gagal/menggantung dan tak ada yang bisa login. Sudah diuji: dengan Redis palsu, sesi tetap aktif di mode file.
- **Batasan jujur (inferensi dari kode)**: sesi lama tidak ikut pindah, jadi pengguna yang sudah login harus login ulang saat Redis mati; login baru bisa, tapi sesinya tersimpan di file lokal.

2. "Password dicek dengan bcrypt, tidak pernah disimpan polos"

- Saat user dibuat/ganti password: `password_hash($pw, PASSWORD_BCRYPT)` → di DB hanya hash `$2y$10$...` (60 karakter, salt acak, sengaja lambat). Password asli tidak ada di DB, sesi, maupun log.
- Saat login: `password_verify($pw, hash)` — hash tidak bisa "dibalik"; dibandingkan, bukan didekripsi. Salah/akun nonaktif/email tak dikenal → pesan sama.

Versi lebih lengkap: Lampiran A.



The screenshot shows a web application titled "Inventory & Order Management" with a subtitle "Sign in to manage inventory and orders". The login form has two input fields: "EMAIL ADDRESS" with the value "admin@example.com" and "PASSWORD" with the placeholder "Enter your password". A red error message at the top of the form states: "The email address or password you entered is incorrect." Below the form is a "Sign In" button. At the bottom, there is a section for "DEMO ACCOUNTS (1 PER ROLE)" with three rows: "ADMIN" (admin@example.com, admin123), "SALES" (sales@example.com, sales123), and "WAREHOUSE STAFF" (warehouse@example.com, wh123). A note at the bottom says "Click an account to fill the form, then Sign In."

Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

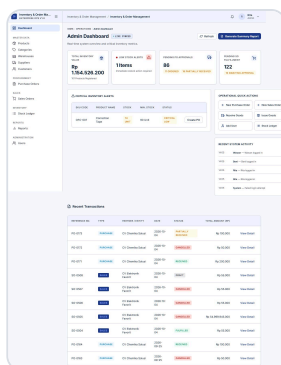
- Ketik password salah → muncul pesan umum *"The email address or password you entered is incorrect."* (tidak menyebut mana yang salah).

🎤 KATAKAN (baca naskah ini)

"Kalau saya salah memasukkan password, pesannya sengaja dibuat umum — tidak memberi tahu mana yang salah."

💡 Kenapa pesan umum — ucapkan di momen ini

Supaya penyerang tidak bisa menebak email mana yang terdaftar (anti user-enumeration). Percobaan gagal juga dicatat untuk audit (event `login_failed` ; email yang dicoba sengaja tidak ikut disimpan di log). Akun nonaktif mendapat pesan yang sama persis.



Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

- Tampilkan dashboard Admin: ringkasan stok, pesanan, aktivitas.
- Tunjuk menu kiri LENGKAP: Dashboard, Master Data (Products, Categories, Warehouses, Suppliers, Customers), Procurement, Sales, Inventory, Reports, Administration (Users).

🎤 KATAKAN (baca naskah ini)

"Saya masuk sebagai Admin. Dashboard menampilkan ringkasan kondisi gudang dan pesanan. Perhatikan menu di kiri — Admin punya akses penuh ke semua modul."

💡 Selip alasan — Role & otorisasi — ucapkan di momen ini

"Menu yang muncul ditentukan izin role saya. Tapi menyembunyikan menu saja bukan keamanan — setiap halaman tetap dicek di server. Nanti saya tunjukkan kalau role lain memaksa buka menu terlarang, dia tetap ditolak."

Soal cache Memcached — data apa tepatnya?

Angka dashboard (nilai inventori, low stock, jumlah PO/SO, antrean) dibaca langsung dari MySQL tiap halaman dibuka (badge "LIVE SYNCED") — **tidak di-cache**. Yang di-cache Memcached hanya tiga hal:

Key	Isi	TTL	Invalidasi
product:<SKU>	Satu produk per SKU (dipakai API <code>/api/products/{sku}/availability</code>); bukan stok	5 menit	Dihapus saat produk diubah / dinonaktifkan / diaktifkan
role_permissions.grouped	Tabel izin role (menu & guard akses)	1 jam	Hanya kedaluwarsa TTL
file_validation.rules	Aturan tipe gambar upload produk	1 jam	Hanya kedaluwarsa TTL

Stok tidak pernah di-cache (selalu MySQL + `SELECT ... FOR UPDATE`) agar anti-oversell tak bergantung data usang. Bila Memcached mati, aplikasi tetap jalan penuh dari MySQL. Versi lengkap: Lampiran B.

SKU CODE	PRODUCT NAME	UOM	PRICE (Rp)	TOTAL PHYSICAL STOCK	STATUS
QTC-001	A4 Paper 80mm Office Supplies	sheet	Rp 55,000	282 sheet	IN STOCK
QAC-002	Acrylic Sheet 3mm Raw Materials	sheet	Rp 145,000	80 sheet	IN STOCK
QPF-003	Air Filter Regulator Spare Parts	pcs	Rp 105,000	88 pcs	IN STOCK
QAR-004	Aluminum Rod 1m Raw Materials	pcs	Rp 140,000	80 pcs	IN STOCK
QAS-005	Aluminum Sheet 3mm Raw Materials	sheet	Rp 295,000	90 sheet	IN STOCK
QBB-006	Ball Bearing 6005 Spare Parts	pcs	Rp 48,000	80 pcs	IN STOCK
QBP-007	Ballpoint Pen Box Office Supplies	box	Rp 40,000	80 box	IN STOCK
QBR-008	Bearing 6004 Spare Parts	pcs	Rp 42,000	77 pcs	IN STOCK
QBC-009	Binder Clip 50mm Box Office Supplies	box	Rp 22,000	80 box	IN STOCK
QES-010	Bluetooth Speaker Electronics	pcs	Rp 299,000	80 pcs	IN STOCK

Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

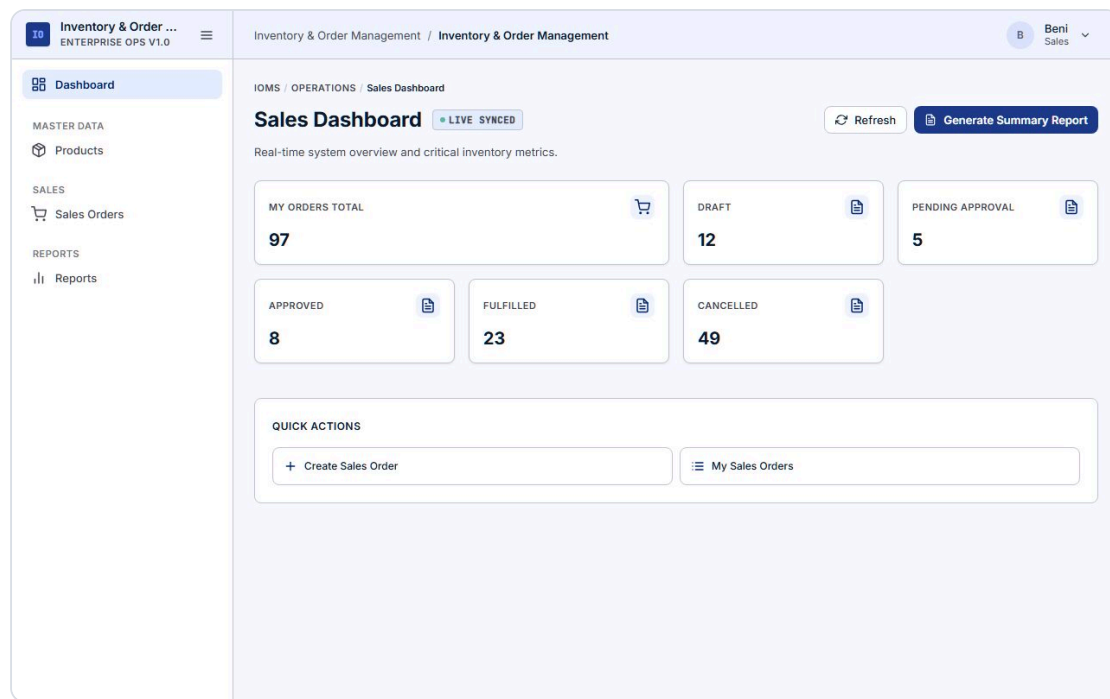
- Buka menu Products (Master Data). Tunjuk daftar produk + ringkasan stok.
- Jelaskan: master data (produk, kategori, gudang, supplier, customer) dikelola Admin — dipakai semua role.

🎤 KATAKAN (baca naskah ini)

"Sebelum ada transaksi, Admin menyiapkan data induk: produk, gudang, supplier, dan customer. Data inilah yang dipakai Sales dan Warehouse nanti."

💡 Selip alasan — Database & soft-delete — ucapkan di momen ini

"Semua data ini tersimpan di MySQL — satu sumber kebenaran. Saya pilih database relasional karena datanya saling terhubung dan butuh transaksi. Dan kalau data induk 'dihapus', sebenarnya hanya dinonaktifkan, bukan hilang — supaya pesanan lama yang merujuknya tidak rusak." (Pengecualian terverifikasi: Kategori punya aksi hapus sungguhan, tetapi ditolak bila masih dipakai produk.)



Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

- Login sebagai Sales (Beni). Tunjuk sidebar: hanya **Dashboard, Products (lihat saja), Sales Orders, Reports**.
- Dashboard Sales hanya menampilkan pesanan miliknya.

🎤 KATAKAN (baca naskah ini)

"Sekarang saya pindah peran menjadi Sales. Lihat bedanya — menunya jauh lebih sedikit. Sales fokus pada Sales Order, dan dashboardnya hanya menampilkan pesanan milik Sales ini sendiri."

💡 Selip alasan — Role journey — ucapkan di momen ini

"Inilah pembagian tugas: Sales fokus membuat pesanan. Data pesanan difilter berdasarkan ID user, jadi Sales tidak melihat pesanan orang lain."

Inventory & Order Management / Inventory & Order Management

Beni (upd) (upd) Sales

— Back **New Sales Order**

ORDER DATE: 04/10/2026

CUSTOMER: — Select — SOURCE WAREHOUSE: — Select —

NOTE:

Line Items

NAME	SKU	UNIT	QTY	SALE PRICE
—			1	0.00

Add Line

Cancel Save

Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

- Menu Sales Orders → tombol buat SO baru (atau Quick Action "Create Sales Order"). Pilih customer, gudang sumber, tanggal.
- Tambah baris barang: pilih produk, isi jumlah & harga. Simpan → jadi Draft.

🔊 KATAKAN (baca naskah ini)

"Tugas utama Sales: membuat pesanan. Saya pilih customer, gudang asal barang, lalu tambah barang beserta jumlah dan harga, lalu Simpan. Pesanan tersimpan sebagai Draft."

💡 Selip alasan — Transaksi — ucapkan di momen ini

"Saat saya tekan Simpan, data dikirim ke server, dicek dulu aturannya — customer dan gudang harus aktif, produk aktif, jumlah minimal 1. Lalu header pesanan dan semua barisnya disimpan dalam satu transaksi, supaya tidak ada pesanan dengan barang separuh. Perhatikan: stok belum dikurangi di sini — stok baru berubah nanti saat barang benar-benar dikeluarkan."

Inventory & Order Management / Inventory & Order Management

IOMS / SALES ORDERS / SO-0508

SO-0508 DRAFT

CV Elektronik Favorit - Order Date: 2026-10-04 - Created by Beni (upd) (upd)

1 Draft — 2 Pending Approval — 3 Approved — 4 Fulfilled

Sales Order & Stock Allocation Protocol. Approved Sales Orders place a hard stock reservation against the assigned Source Warehouse. Physical inventory is deducted from ProductStock only upon validated Goods Issue, recording an immutable audit entry in the Stock Ledger.

CUSTOMER
CV Elektronik Favorit
Customer master record verified

SOURCE FACILITY
Bandung Warehouse
Physical inventory deducted here upon Goods Issue.

AUDIT & CHAIN OF CUSTODY
Order Date: 2026-10-04
Stock Ledger Entries: 0

Line Items 1 LINE

PRODUCT	SKU	QTY	UNIT PRICE	LINE SUBTOTAL
A4 Paper Ream ream	0FC-001 (EDITED) (EDITED) (EDI	1	Rp 55.000	Rp 55.000
Distinct SKUs: 1 - Total Quantity: 1 unit				Rp 55.000

Submit for Approval Cancel Order Back

Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

- Buka detail SO yang Draft. Tunjuk tombol **Submit for Approval** (juga ada **Edit Draft** dan **Cancel Order**).
- Klik Submit → status berubah ke "Pending Approval".

🎤 KATAKAN (baca naskah ini)

"Setelah pesanan siap, saya klik Submit. Statusnya berubah menjadi Menunggu Persetujuan. Sekarang pesanan ini keluar dari tangan Sales dan menunggu Admin — ini perpindahan pertama antar-role."

💡 Selip alasan — Handoff — ucapkan di momen ini

"Sales tidak bisa menyetujui pesannya sendiri. Jadi di titik ini tugas berpindah ke Admin."

Inventory & Order Management / Inventory & Order Management

Beni (upd) (upd) Sales

Dashboard

MASTER DATA

Products

SALES

Sales Orders

REPORTS

Reports

Back New Sales Order

ORDER DATE

04/10/2026

CUSTOMER

— Select —

SOURCE WAREHOUSE

— Select —

NOTE

Line Items

NAME	SKU	UNIT	QTY	SALE PRICE
—			1	0.00

Add Line

Cancel Save

Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

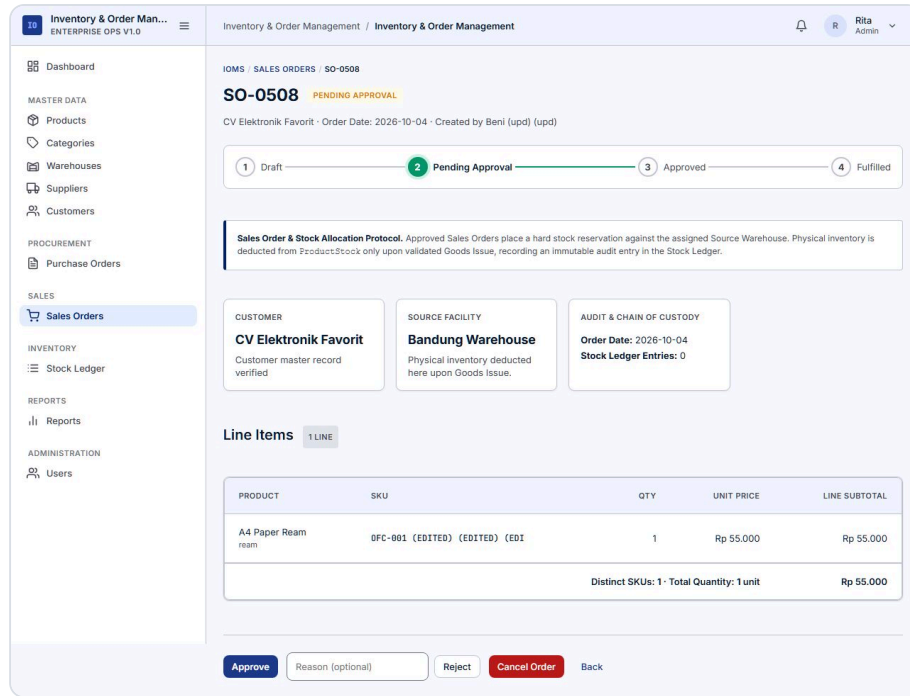
- Masih sebagai Sales, buka pesanan yang Pending. Tunjuk: TIDAK ada tombol Approve/Reject (hanya Cancel Order).

🎤 KATAKAN (baca naskah ini)

"Perhatikan baik-baik: sebagai Sales, di pesanan yang menunggu persetujuan ini, saya tidak punya tombol Approve. Ini sengaja."

💡 Selip alasan — Segregation of Duties — ucapkan di momen ini

"Inilah Segregation of Duties — Sales tidak boleh menyetujui pesanan apa pun, termasuk miliknya sendiri. Ini aturan anti-kecurangan, dan ditegakkan di server lewat tabel izin `role_permissions` dan `SalesOrderPolicy`, bukan sekadar menyembunyikan tombol."



Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

- Login sebagai Admin. Buka pesanan yang sama (Pending Approval).
- Sekarang MUNCUL tombol Approve / Reject. Klik Approve → status jadi Approved.

🎤 KATAKAN (baca naskah ini)

"Saya kembali menjadi Admin, dan membuka pesanan yang tadi. Lihat — sekarang tombol Approve dan Reject muncul, karena hanya Admin yang berhak. Saya klik Approve, dan statusnya menjadi Disetujui. Dicatat juga siapa yang menyetujui dan kapan, untuk audit."

💡 Selip alasan — Otorisasi server — ucapkan di momen ini

"Jadi role berbeda, pesanan yang sama persis — tapi tombolnya berbeda tergantung siapa yang login. Itu bukti otorisasi bekerja di level server. Sekarang pesanan siap diserahkan ke Warehouse untuk dikeluarkan barangnya." (Catatan: Reject mengubah status menjadi Cancelled dengan alasan opsional — tidak ada status "Rejected" terpisah.)

The screenshot displays the 'Warehouse Dashboard' interface. It includes a sidebar with navigation links: Dashboard, Products, Purchase Orders, Stock Ledger, and Reports. The main content area features a 'Warehouse Dashboard' header with a 'Refresh' button and a 'Generate Summary Report' button. Below this, there are two summary cards: 'GOODS RECEIPT QUEUE' with a value of 29 and 'GOODS ISSUE QUEUE' with a value of 15. A 'BELOW REORDER POINT' indicator shows a value of 1. The dashboard is divided into two main sections: 'Goods Issue Queue — Approved Sales Orders' and 'Goods Receipt Queue — Purchase Orders Awaiting Delivery'. Each section contains a table with columns for ID, DATE, CUSTOMER, SOURCE WAREHOUSE, LINES, and a status (e.g., 'Process Issue' or 'Receive'). The 'Goods Issue Queue' table lists 12 items, and the 'Goods Receipt Queue' table lists 12 items. At the bottom, there is a 'Low Stock Products' section with a table showing items like 'OPIC 007' with a 'Current Stock' of 74 and a 'Reorder Point' of 90.

Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

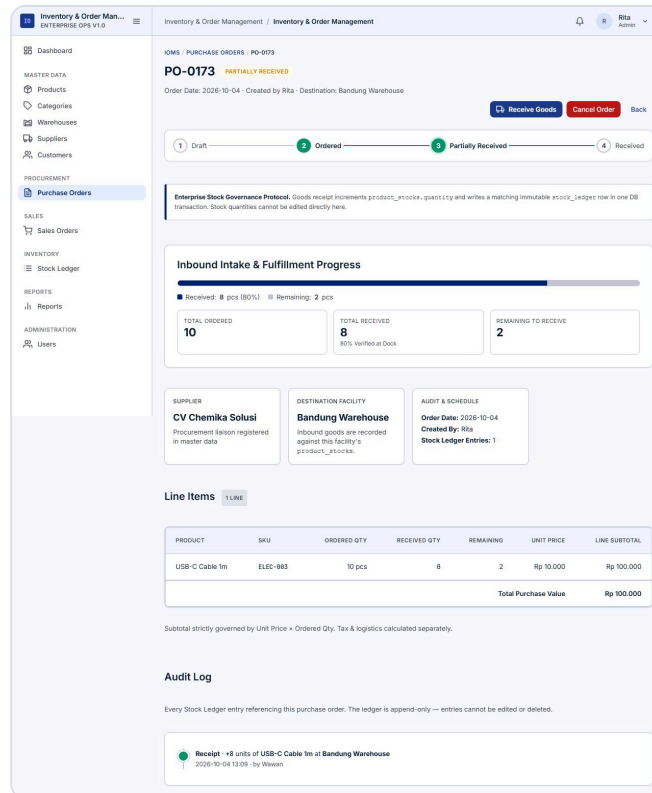
- Login sebagai Warehouse (Wawan). Sidebar: **Dashboard, Products (lihat saja), Purchase Orders, Stock Ledger, Reports**. Tidak ada menu Sales Orders.
- Pintu masuk ke pengeluaran barang adalah Dashboard: tabel **Goods Issue Queue** (SO Approved, tombol **Process Issue**) dan **Goods Receipt Queue** (PO menunggu barang, tombol **Receive**).

🎤 KATAKAN (baca naskah ini)

"Sekarang saya menjadi Warehouse Staff. Tugas saya menangani barang fisik: menerima barang dari pembelian, dan mengeluarkan barang untuk pesanan yang sudah disetujui. Antrean kerjanya langsung ada di dashboard."

💡 Selip alasan — Kartu Stok — ucapkan di momen ini

"Di sinilah stok benar-benar berubah. Setiap pergerakan barang dicatat di Kartu Stok — aplikasi tidak menyediakan fitur untuk mengedit atau menghapus barisnya — supaya selalu bisa diaudit."



Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

- Buka Purchase Order berstatus Ordered (dari antrean dashboard → **Receive**, atau Purchase Orders → detail → **Receive Goods**). Isi jumlah diterima per baris, simpan.
- Tunjuk: stok produk NAIK, muncul baris di Kartu Stok bertipe Receipt.

🎤 KATAKAN (baca naskah ini)

"Pertama, menerima barang dari pemasok lewat Purchase Order. Saya isi jumlah yang diterima lalu simpan. Stok produk naik, dan muncul catatan baru di Kartu Stok."

💡 Selip alasan — Transaksi — ucapkan di momen ini

"Penambahan stok dan pencatatan Kartu Stok terjadi dalam satu transaksi — kalau salah satu gagal, dua-duanya dibatalkan, jadi stok dan catatannya tidak pernah beda."

The screenshot shows a 'Stock Ledger' application window. It has a sidebar with navigation options like 'Dashboard', 'Inventory', 'Stock Ledger', 'Reports', 'Settings', and 'Users'. The main area displays a table of stock transactions. The table has columns for 'DATE', 'PRODUCT NAME', 'QUANTITY', 'UNIT PRICE', 'TOTAL PRICE', 'CURRENCY', and 'STATUS'. The transactions are listed in a table with alternating light and dark blue rows. The status of each transaction is indicated by a colored dot: green for 'Selesai' (Completed) and red for 'Masuk' (Incoming).

DATE	PRODUCT NAME	QUANTITY	UNIT PRICE	TOTAL PRICE	CURRENCY	STATUS
2023-10-25	Produk A	10	1000	10000	Rp	Selesai
2023-10-25	Produk B	5	2000	10000	Rp	Masuk
2023-10-25	Produk C	15	500	7500	Rp	Selesai
2023-10-25	Produk D	8	1250	10000	Rp	Masuk
2023-10-25	Produk E	12	833	10000	Rp	Selesai
2023-10-25	Produk F	10	1000	10000	Rp	Masuk
2023-10-25	Produk G	15	666	10000	Rp	Selesai
2023-10-25	Produk H	10	1000	10000	Rp	Masuk
2023-10-25	Produk I	12	833	10000	Rp	Selesai
2023-10-25	Produk J	10	1000	10000	Rp	Masuk
2023-10-25	Produk K	15	666	10000	Rp	Selesai
2023-10-25	Produk L	10	1000	10000	Rp	Masuk
2023-10-25	Produk M	12	833	10000	Rp	Selesai
2023-10-25	Produk N	10	1000	10000	Rp	Masuk
2023-10-25	Produk O	15	666	10000	Rp	Selesai
2023-10-25	Produk P	10	1000	10000	Rp	Masuk
2023-10-25	Produk Q	12	833	10000	Rp	Selesai
2023-10-25	Produk R	10	1000	10000	Rp	Masuk
2023-10-25	Produk S	15	666	10000	Rp	Selesai
2023-10-25	Produk T	10	1000	10000	Rp	Masuk
2023-10-25	Produk U	12	833	10000	Rp	Selesai
2023-10-25	Produk V	10	1000	10000	Rp	Masuk
2023-10-25	Produk W	15	666	10000	Rp	Selesai
2023-10-25	Produk X	10	1000	10000	Rp	Masuk
2023-10-25	Produk Y	12	833	10000	Rp	Selesai
2023-10-25	Produk Z	10	1000	10000	Rp	Masuk

Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

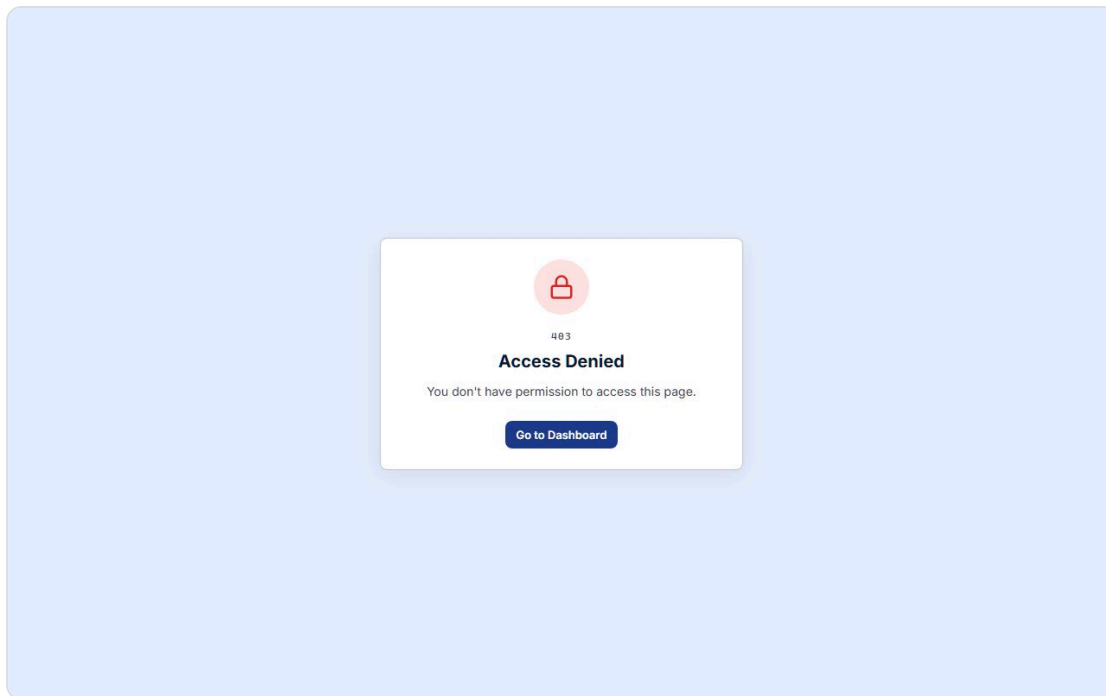
- Dari Goods Issue Queue klik **Process Issue** pada SO Approved → klik **Issue Goods**. Stok TURUN, status jadi Fulfilled.
- Buka Stock Ledger (Kartu Stok): tunjuk baris Receipt (masuk) & Issue (keluar) — bukti jejak lengkap.

🎤 KATAKAN (baca naskah ini)

"Dan ini puncaknya. Untuk pesanan yang sudah disetujui, saya klik Issue — barang keluar, stok turun, status menjadi Selesai. Kalau saya buka Kartu Stok, terlihat seluruh jejaknya: barang masuk waktu diterima, dan keluar waktu dikeluarkan."

💡 Selip alasan — Anti-oversell (penting!) — ucapkan di momen ini

"Di balik layar, saat barang dikeluarkan, baris stok dikunci dulu (SELECT ... FOR UPDATE) supaya dua pengeluaran bersamaan tidak bisa mengambil stok yang sama. Kalau stok tidak cukup, pengeluaran ditolak dan stok tidak jadi minus. Jadi oversell dicegah secara teknis — ini salah satu syarat kritikal project ini."



Tampilan aplikasi yang harus Anda tampilkan

► TAMPILKAN / LAKUKAN

- Sebagai Sales atau Warehouse, ketik langsung URL menu terlarang `/users`. Muncul halaman 403 ("You don't have permission to access this page.").
- Variasi lain: Sales buka `/purchase-orders` atau `/stock-ledger` → 403.

🎤 KATAKAN (baca naskah ini)

"Terakhir, bukti keamanannya. Kalau role yang tidak berhak memaksa membuka menu terlarang dengan mengetik alamatnya langsung, dia tetap ditolak dengan halaman 403. Jadi keamanan bukan sekadar menyembunyikan menu — dicek sungguhan di server."

💡 Selip alasan — Keamanan server — ucapkan di momen ini

"Menyembunyikan tombol di tampilan bukan otorisasi; pengecekan sesungguhnya ada di server pada setiap permintaan." *Catatan jujur: tidak semua halaman daftar dijaga permission — lihat Lampiran F.*

Langkah bicara / persiapan — tidak perlu layar tertentu

► TAMPILKAN / LAKUKAN

- Rangkum perjalanan: Sales buat → Admin setuju → Warehouse keluarkan. Siap masuk tanya jawab.

🎤 KATAKAN (baca naskah ini)

"Jadi tadi satu pesanan berjalan lengkap: Sales membuatnya, Admin menyetujuinya, Warehouse mengeluarkan barangnya — dan di setiap langkah, aplikasi menjaga siapa yang berhak, mencatat jejaknya, dan memastikan stok tidak pernah minus. Itulah inti dari Inventory & Order Management System ini. Saya siap untuk pertanyaan."

💡 Jika ditanya "kenapa Redis DAN Memcached" — ucapkan di momen ini

Jujur: Redis untuk sesi (butuh TTL), Memcached untuk cache baca. Keduanya sebenarnya tidak wajib — saya pakai untuk mendemonstrasikan externalized session & cache-aside, dengan peran yang dipisah jelas (ADR-005). Lihat Lampiran A dan B.

Menjabarkan kalimat di Langkah 1: *"Kalau Redis kebetulan mati, aplikasi otomatis beralih ke penyimpanan file, jadi login tetap jalan."* dan *"password dicek dengan bcrypt, tidak pernah disimpan polos."* Sumber: `app/Core/SessionManager.php`, `app/Service/AuthService.php`, `app/Service/UserService.php`, ADR-005. Fallback juga diuji langsung saat penyusunan dokumen ini (lihat A.3).

A.1 Apa itu "sesi" dan kenapa di Redis?

- Setelah login berhasil, server menyimpan `user_id`, `role`, `user_name`, dan `last_activity` di sesi PHP. Browser hanya memegang *cookie* berisi ID sesi acak (`iom_session`) — bukan data pengguna.
- Penyimpanan sesi default PHP adalah file di server. Aplikasi ini menggantinya ke **Redis** (`session.save_handler=redis`, alamat `tcp://redis:6379?database=0`) agar: (1) sesi kedaluwarsa otomatis oleh Redis (TTL), (2) cepat dibaca di tiap halaman karena di memori, (3) tidak membebani MySQL, dan (4) tidak terikat satu server/kontainer.
- Redis **hanya** untuk sesi. Data bisnis (termasuk stok) tidak pernah disimpan di Redis; sesi bisa hilang tanpa merusak data — dampaknya cuma pengguna harus login ulang (ADR-005).
- Masa berlaku: `SESSION_LIFETIME = 3600` detik (default di `docker-compose.yml`). Selain cookie, server juga memeriksa **idle timeout**: bila selisih `last_activity` dengan sekarang melebihi lifetime, sesi dihancurkan dan pengguna dianggap belum login (`AuthService::currentUser()`).
- Cookie sesi diamankan: `HttpOnly` (tidak terbaca JavaScript), `SameSite=Lax`, dan flag `Secure` aktif di produksi bila lewat HTTPS (termasuk di balik reverse proxy Caddy via header `X-Forwarded-Proto`).
- Saat login ID sesi diganti (`session_regenerate_id(true)`) untuk mencegah *session fixation*, dan token CSRF dibuat baru.

A.2 Bagaimana fallback "Redis mati → file" bekerja

1. Setiap request, `SessionManager::start()` melakukan **probe TCP** singkat ke Redis (`fsockopen`, timeout 0,5 detik). Ini hanya cek "port terbuka?", bukan handshake Redis penuh.
2. Bila terjangkau → sesi disimpan di Redis. Bila tidak → aplikasi menulis peringatan ke error log (`"Redis unreachable ... falling back to file sessions"`) dan memakai `session.save_handler=files`.
3. Tujuannya (kode ditandai C-02): tanpa fallback, handler Redis yang mati membuat **setiap** request gagal atau menggantung, sehingga tak seorang pun bisa login. Dengan fallback, halaman login tetap berfungsi, pengguna bisa masuk, dan aplikasi tetap dipakai.

Batasan yang perlu diketahui (disimpulkan dari logika kode — inferensi)

Pilihan handler dibuat **per request** dan sesi tidak dipindahkan antar penyimpanan. Akibatnya: pengguna yang sudah login lewat Redis akan terlihat "logout" saat Redis mati (sesinya ada di Redis, sementara request baru membaca file), dan harus login ulang; login baru pada masa gangguan tersimpan di file lokal kontainer dan tidak terlihat lagi setelah Redis pulih. Probe hanya mengecek TCP, jadi Redis yang port-nya terbuka tetapi macet tidak terdeteksi. Jawaban jujur saat ditanya: "login tetap bisa, tetapi sesi lama tidak ikut pindah."

A.3 Bukti verifikasi

Dijalankan saat penyusunan dokumen: `SessionManager` diarahkan ke Redis palsu (`127.0.0.1:6399`). Hasil: log "falling back to file sessions", `session.save_handler=files`, sesi aktif dan data sesi dapat ditulis/dibaca. Pada mode normal (Redis hidup) handler = `redis`. Untuk demo langsung: `docker compose stop redis` lalu login → tetap berhasil; `docker compose start redis` untuk memulihkan.

A.4 Password: bcrypt, tidak pernah polos

- **Menyimpan:** saat Admin membuat user atau mengganti password, aplikasi memanggil `password_hash($password, PASSWORD_BCRYPT) ( UserService )`. Yang masuk kolom `users.password_hash` hanyalah hash berawalan `$2y$10$...` (60 karakter; `$2y$` = bcrypt PHP, `10` = cost). Password asli tidak disimpan di database, sesi, maupun log. Data seed juga berisi hash, bukan teks polos.
- **Mengecek:** saat login, `password_verify($password, $user->passwordHash)` membandingkan input dengan hash. Hash tidak dapat "dibalik" ke password; bcrypt memakai *salt* acak per password (tertanam dalam hash) sehingga dua user dengan password sama punya hash berbeda, dan sengaja lambat (cost $10 = 2^{10}$ putaran) agar sulit ditebak massal.
- Akun nonaktif, email tak dikenal, dan password salah menghasilkan pesan yang sama (anti user-enumeration); percobaan gagal dicatat sebagai `login_failed`.
- Aturan password: minimal 6 karakter saat dibuat Admin; email unik. Tidak ada ganti password mandiri — Admin menggantinya lewat Users → Edit (My Profile hanya nama & email).

Koreksi penting

Kalimat di versi sebelumnya — "sebagian data dashboard yang sering dibaca & jarang berubah dipercepat dengan cache Memcached" — **tidak sesuai kode**. `DashboardService` dan `SalesDashboardService` tidak memakai cache sama sekali; seluruh angka dashboard (nilai inventori, low stock, jumlah PO/SO per status, antrean) dibaca langsung dari MySQL tiap kali halaman dibuka. Kalimat itu sudah diganti di Langkah 2.

Hanya **tiga** jenis data yang di-cache (pola cache-aside lewat `CacheService`; sumber: `ProductService`, `PermissionService`, `FileValidationService`, ADR-005):

Data (key)	Isi & dipakai di mana	TTL	Invalidasi
<code>product:<SKU></code> (SKU huruf besar)	Satu produk hasil pencarian per SKU (<code>ProductService::findBySku</code>). Dipakai mis. oleh endpoint <code>GET /api/products/{sku}/availability</code> . Hanya data produk — bukan stok .	300 detik (5 menit)	Dihapus (<code>delete</code>) saat produk diubah, dinonaktifkan atau diaktifkan (<code>invalidateSkuCache</code>); bila SKU berubah, key lama dan baru dihapus.
<code>role_permissions.grouped</code>	Seluruh tabel izin role (siapa boleh apa), dibaca sekali lalu dipakai untuk menentukan menu sidebar dan guard <code>requirePermission()</code> di tiap request.	3600 detik (1 jam)	Tidak ada invalidasi manual — hanya kedaluwarsa TTL. Mengubah isi <code>role_permissions</code> baru berlaku maksimal ± 1 jam kemudian (atau saat Memcached di-restart).
<code>file_validation.rules</code>	Aturan tipe gambar yang diizinkan saat upload produk (ekstensi, MIME, byte header/footer) dari tabel <code>file_validation_rules</code> .	3600 detik (1 jam)	Hanya TTL. Bila hasil kosong, upload ditolak (bukan diizinkan semua).

Yang sengaja TIDAK di-cache

- **Stok** (`product_stocks`) dan ketersediaan — selalu dibaca langsung dari MySQL; penjaga konsistensinya adalah transaksi + `SELECT ... FOR UPDATE` (ADR-002), bukan cache. Alasannya agar syarat ARCH-02 (anti-oversell) tidak pernah bergantung pada data usang.
- Dashboard, daftar produk/SO/PO, laporan, Stock Ledger — semuanya langsung dari database.

Perilaku bila Memcached mati

`CacheService` mengecek koneksi saat dibuat (timeout 500 ms). Jika ekstensi/ server tidak tersedia, ia menandai diri "tidak tersedia" dan semua `get` mengembalikan kosong, `set/delete` tidak melakukan apa-apa — aplikasi tetap berfungsi penuh, hanya membaca dari MySQL. Cache bukan sumber kebenaran dan tidak pernah write-through.

Redis vs Memcached — ringkas untuk tanya-jawab

Redis = sesi login (butuh TTL, hilang tak merusak data). Memcached = cache baca yang bisa dibangun ulang dari MySQL. Dipisah agar kelak tak ada yang "sekalian" menyimpan stok di store yang juga menjaga login.

Admin memegang semua permission key di `role_permissions`. Centang tiap butir saat dites. Format: **Lakukan → Hasil yang diharapkan**. Sumber: `config/routes.php`, `views/layouts/main.php`, controller & view terkait.

C.1 Peta menu & fungsinya

Menu (URL)	Untuk apa
Dashboard <code>/dashboard</code>	Ringkasan live: total nilai inventori & jumlah produk, produk di bawah reorder point, PO per status, SO per status (semua Sales), transaksi terbaru (5 PO + 5 SO), 5 aktivitas audit terakhir, Quick Actions.
Products <code>/products</code>	Katalog produk + stok per gudang; tambah/ubah/nonaktifkan produk.
Categories <code>/categories</code>	Pengelompokan produk; ekspor CSV daftar kategori.
Warehouses <code>/warehouses</code>	Lokasi gudang; detail menampilkan stok per produk di gudang itu.
Suppliers <code>/suppliers</code>	Pemasok untuk Purchase Order.
Customers <code>/customers</code>	Pelanggan untuk Sales Order.
Purchase Orders <code>/purchase-orders</code>	Pembelian ke supplier → menerima barang (menambah stok).
Sales Orders <code>/sales-orders</code>	Seluruh pesanan penjualan; approve/reject; kirim barang (mengurangi stok).
Stock Ledger <code>/stock-ledger</code>	Kartu stok: jejak audit semua pergerakan (Receipt/Issue).
Reports <code>/reports</code>	Analitik inventori + ekspor CSV (ledger, pesanan).
Users <code>/users</code>	Kelola akun dan role pengguna.
My Profile <code>/my-profile</code> (menu akun kanan atas)	Ubah nama & email sendiri; juga tombol Logout.
Bel notifikasi (header)	Peringatan stok rendah (dibuat job terjadwal tiap 15 menit); "Mark All as Read".

C.2 Checklist how-to

Login & Dashboard

- ☐ Login `admin@example.com` → diarahkan ke `/dashboard`; sidebar menampilkan SEMUA grup (Master Data, Procurement, Sales, Inventory, Reports, Administration).
- ☐ Kartu: Total Inventory Value, Below Reorder Point, PO status, SO status ("n Awaiting Approval" bila ada) tampil; klik **Refresh** memuat ulang angka.
- ☐ Daftar Low Stock punya tombol **Create PO** → membuka form PO. Quick Actions: buat PO, buat SO, lihat PO/SO, tambah user (`/users/create`), Stock Ledger.
- ☐ Bel notifikasi: bila ada peringatan, klik **Mark All as Read** → hitungan menjadi 0.

Products

- ☐ Cari/filter (SKU, nama, kategori, gudang, status stok), ubah urutan & jumlah per halaman (10/25/50/100) → daftar menyesuaikan.
- ☐ Add Product: isi SKU (unik), barcode, nama (min. 3 huruf), deskripsi, kategori, unit, harga beli/jual, reorder point, gudang & qty awal (opsional), gambar JPG/PNG/WebP → tersimpan; SKU ganda ditolak dengan pesan validasi.

- ☐ Upload file bukan gambar (mis. .exe diganti nama .jpg) → ditolak (validasi ekstensi + byte signature).
- ☐ Detail produk: stok per gudang dan pergerakan terakhir. Edit → simpan. **Deactivate** → hilang dari daftar default & dropdown PO/SO baru; **Activate** mengembalikan.

Categories / Warehouses / Suppliers / Customers

- ☐ Tiap modul: list + filter + paging; Add (Categories: kode, nama, deskripsi · Warehouses: kode, nama, lokasi · Suppliers: nama, kontak, email · Customers: nama, kontak, email, telepon) → tersimpan.
- ☐ Edit & Deactivate/Activate pada tiap modul → status berubah; data nonaktif tak muncul di dropdown transaksi baru.
- ☐ Categories saja: **Export CSV** (mengikuti filter) dan aksi **hapus sungguhan**; hapus kategori yang masih dipakai produk → ditolak.
- ☐ Warehouses → detail: jumlah produk & total qty di gudang itu, tabel stok per produk dengan paging.

Purchase Orders

- ☐ Create PO: pilih supplier, gudang tujuan, tanggal, catatan, baris produk (qty ≥ 1, harga ≥ 0) → status **Draft**.
- ☐ Detail Draft → **Submit Order** → **Ordered** (hanya Admin). Stepper status Draft→Ordered→PartiallyReceived→Received tampil.
- ☐ **Receive Goods** → isi qty per baris (boleh sebagian; tidak boleh melebihi sisa) → stok gudang tujuan naik, baris Receipt muncul di Stock Ledger, status PartiallyReceived/Received.
- ☐ **Cancel Order** hanya muncul/diterima sebelum ada penerimaan barang; PO yang sudah menerima barang tak bisa dibatalkan.

Sales Orders

- ☐ List menampilkan SO semua Sales; filter status, nomor SO, customer, gudang, urutan.
- ☐ Create/Edit Draft (Admin boleh mengedit Draft siapa pun) → Submit for Approval → **Approve** (tercatat approved_by & waktu) atau **Reject** dengan alasan opsional → status Cancelled.
- ☐ SO Approved → **Issue Goods** → stok gudang asal turun, baris Issue di ledger, status Fulfilled; stok tidak cukup → ditolak, tidak ada perubahan.
- ☐ **Cancel Order** pada SO apa pun yang belum Fulfilled; SO Fulfilled tidak bisa dibatalkan siapa pun.
- ☐ Admin boleh menyetujui SO buatannya sendiri (larangan self-approval hanya untuk Sales).

Stock Ledger & Reports

- ☐ Stock Ledger: filter SKU, nama produk, jenis (Receipt/Issue/Adjustment), gudang;urut kolom; 25 baris/halaman → hasil sesuai. Tidak ada tombol edit/hapus baris.
- ☐ Reports: pilih jenis laporan (Stock Valuation & Turnover, Inventory Aging, Slow-Moving & Dead Stock, Movement Ledger Audit), rentang tanggal, gudang, kategori, pencarian, urutan → **Generate**; grafik & tabel (5 baris/halaman) berubah.
- ☐ **Export Stock Ledger** (CSV, mengikuti filter) dan **Export Orders** (CSV gabungan SO semua Sales + semua PO; wajib isi tanggal mulai & akhir, mulai ≤ akhir); tombol preview menampilkan tabel contoh isi CSV.

Users & Profile

- ☐ Add User: nama, email (unik, tak peka huruf besar/kecil), role (Admin/Sales/WarehouseStaff), password (min. 6) → bisa langsung login.
- ☐ Edit user: password dikosongkan = tidak berubah. Deactivate → user itu tidak bisa login & sesi aktifnya ditolak pada request berikutnya; Activate memulihkan.
- ☐ Admin tidak bisa menonaktifkan akunnya sendiri atau mengubah role-nya sendiri → ditolak.
- ☐ My Profile: ubah nama/email → muncul pesan sukses. Logout (menu akun) → kembali ke login

Permission Sales: `customers.view`, `sales_orders.menu`, `reports.sales_orders.view`. Tidak punya semua permission `*.manage`, `approve`, `issue`, `stock ledger`.

D.1 Peta menu & fungsinya

Menu (URL)	Untuk apa
Dashboard <code>/dashboard</code>	Jumlah pesanan milik sendiri : total + per status (Draft, Pending Approval, Approved, Fulfilled, Cancelled); Quick Actions "Create Sales Order" & "My Sales Orders". Tanpa bel notifikasi.
Products <code>/products</code> (read-only)	Melihat katalog, harga jual, dan stok untuk menyusun pesanan. Tanpa tombol tambah/ubah.
Sales Orders <code>/sales-orders</code>	Fitur utama: membuat, mengedit Draft, mengajukan, membatalkan, dan memantau pesanan miliknya.
Reports <code>/reports</code>	Hanya ekspor CSV pesanan miliknya (halaman versi ringkas: rentang tanggal + Export Orders).
My Profile	Ubah nama & email sendiri.

Catatan: menu **Customers tidak ada di sidebar Sales** (link hanya muncul bila punya `master_data.menu`, milik Admin). Namun Sales punya `customers.view`, jadi `/customers` bisa dibuka lewat URL (read-only). Customer dipilih lewat dropdown di form SO. Halaman `/sales-dashboard` (ringkasan dengan pemilih periode) juga ada dan dijaga `sales_orders.menu`, tetapi tidak ditemukan tautan menuju halaman itu di tampilan — **inferensi** dari pencarian view.

D.2 Checklist how-to

- ☐ Login `sales1@example.com` → dashboard Sales; sidebar hanya Dashboard, Products, Sales Orders, Reports.
- ☐ Products: cari/filter; tidak ada tombol Add/Edit/Deactivate. Buka `/products/create` langsung → 403.
- ☐ Sales Orders → list hanya berisi SO buatan sendiri; filter status/nomor/customer/gudang bekerja.
- ☐ Create Sales Order: pilih customer aktif, gudang asal aktif, tanggal, baris produk ($qty \geq 1$, $harga \geq 0$) → status **Draft**; stok tidak berubah.
- ☐ Simpan tanpa baris item, atau dengan $qty = 0$ → ditolak dengan pesan validasi.
- ☐ Detail Draft: tombol **Edit Draft** (ubah & simpan), **Submit for Approval**, **Cancel Order**.
- ☐ Submit → **Pending Approval**; sekarang tombol Edit/Submit hilang, tombol Approve/Reject tidak ada, Cancel Order masih ada.
- ☐ Cancel pada SO Draft/Pending milik sendiri → konfirmasi "Cancel this sales order?" → status Cancelled. Pada SO Approved/Fulfilled → tombol Cancel tidak tersedia (minta Admin).
- ☐ Setelah Admin approve: status Approved; tidak ada tombol Issue Goods untuk Sales. Setelah Warehouse issue: Fulfilled.
- ☐ Reports: pilih rentang tanggal → **Export Orders** → CSV hanya berisi pesanan Sales ini; tanggal kosong atau mulai > akhir → ditolak.
- ☐ My Profile: ubah nama/email → sukses. Logout.

D.3 Pemeriksaan negatif (harus ditolak)

Coba

Hasil yang benar

Ketik <code>/users</code> , <code>/purchase-orders</code> , <code>/stock-ledger</code>	403
Buka detail SO milik Sales lain (<code>/sales-orders/{id}</code>)	403
POST langsung ke <code>/sales-orders/{id}/approve</code> atau <code>/reject</code> , termasuk untuk SO sendiri	Ditolak: "Only an administrator can approve a sales order."
POST ke <code>/sales-orders/{id}/issue</code>	403
Export Stock Ledger (<code>/reports/export/stock-ledger</code>)	403
Buka <code>/categories</code> , <code>/warehouses</code> , <code>/suppliers</code>	Tampil read-only (HTTP 200, tanpa tombol kelola) — halaman daftar hanya butuh login; aksi create/edit/deactivate tetap 403.

Permission Warehouse: `purchase_orders.manage`, `sales_orders.issue`, `stock_ledger.view`, `reports.stock_ledger.view`. Tidak punya submit/cancel PO, approve SO, `sales_orders.menu`, `customers.view`.

E.1 Peta menu & fungsinya

Menu (URL)	Untuk apa
Dashboard <code>/dashboard</code>	Antrean kerja gudang: Goods Receipt Queue (PO Ordered + PartiallyReceived, tombol Receive), Goods Issue Queue (SO Approved, tombol Process Issue; tertua dulu, 10 teratas), jumlah & daftar produk di bawah reorder point, bel notifikasi stok rendah.
Products <code>/products</code> (read-only)	Cek stok per gudang untuk menentukan restock.
Purchase Orders <code>/purchase-orders</code>	Melihat PO, membuat PO usulan restock (Draft), dan menerima barang .
Stock Ledger <code>/stock-ledger</code>	Audit/rekonsiliasi pergerakan stok fisik vs sistem.
Reports <code>/reports</code>	Analitik inventori + Export Stock Ledger (CSV).
My Profile	Ubah nama & email sendiri.

Catatan: tidak ada menu **Sales Orders** di sidebar. Jalur resmi untuk mengeluarkan barang adalah Dashboard → Process Issue → detail SO → **Issue Goods** (kodenya memang menjelaskan antrean dashboard sebagai satu-satunya jalur). Meski tanpa menu, `/sales-orders` dan detailnya tetap bisa dibuka lewat URL oleh Warehouse (hanya butuh login; Warehouse melihat semua SO).

E.2 Checklist how-to

- ☐ Login `warehouse@example.com` → dashboard Warehouse; sidebar: Dashboard, Products, Purchase Orders, Stock Ledger, Reports; bel notifikasi tampil.
- ☐ Dashboard: angka Goods Receipt Queue & Goods Issue Queue cocok dengan jumlah PO Ordered/PartiallyReceived dan SO Approved; bila ada notifikasi, **Mark All as Read** bekerja.
- ☐ Products: lihat stok per gudang; tidak ada tombol Add/Edit.
- ☐ Purchase Orders: list + filter status/nomor/supplier; detail menampilkan progres penerimaan dan baris ledger terkait.
- ☐ Create PO (usulan restock) → tersimpan **Draft**; di detail Draft tidak ada tombol Submit Order dan tidak ada Cancel Order (Admin saja).
- ☐ PO Ordered/PartiallyReceived → **Receive Goods**: isi qty diterima per baris (bisa sebagian) → stok naik, baris Receipt di Stock Ledger, status PartiallyReceived/Received. Qty melebihi sisa → ditolak.
- ☐ Dashboard → **Process Issue** pada SO Approved → detail SO hanya menampilkan **Issue Goods** (tidak ada Approve/Reject) → stok turun, baris Issue di ledger, status Fulfilled.
- ☐ Issue dengan stok gudang asal kurang → ditolak "insufficient stock", stok & status tidak berubah.
- ☐ Stock Ledger: filter SKU/nama/jenis/gudang,urut kolom, 25 baris/halaman; baris Receipt & Issue yang baru dibuat muncul; tidak ada fitur edit/hapus.
- ☐ Reports: generate analitik; **Export Stock Ledger** → CSV. Tombol Export Orders: Warehouse tak punya izin ekspor SO/PO → 403 bila dipanggil langsung.
- ☐ My Profile: ubah nama/email → sukses. Logout.

E.3 Pemeriksaan negatif (harus ditolak)

Coba	Hasil yang benar
Ketik <code>/users</code>	403
Ketik <code>/customers</code>	403 (butuh <code>customers.view</code>)
Buka <code>/sales-orders/create</code> atau POST buat SO	403 (butuh <code>sales_orders.menu</code>)
POST <code>/sales-orders/{id}/approve</code> atau <code>/reject</code>	403
POST <code>/purchase-orders/{id}/submit</code> atau <code>/cancel</code>	403
Create/edit/deactivate produk, kategori, gudang, supplier, customer	403
Buka <code>/sales-dashboard</code>	403

F.1 Matriks permission (dari `database/seed.sql`)

Permission	Admin	Sales	Warehouse
users.manage, products/categories/warehouses/suppliers/customers.manage, master_data.menu	✓	✗	✗
customers.view	✓	✓	✗
purchase_orders.manage (lihat, buat, receive)	✓	✗	✓
purchase_orders.submit / .cancel	✓	✗	✗
sales_orders.menu (lihat menu, buat/edit SO)	✓	✓	✗
sales_orders.approve	✓	✗	✗
sales_orders.issue	✓	✗	✓
stock_ledger.view, reports.stock_ledger.view	✓	✗	✓
reports.sales_orders.view	✓	✓ (milik sendiri)	✗
reports.purchase_orders.view	✓	✗	✗

F.2 Koreksi yang dilakukan pada panduan ini

Versi lama	Fakta terverifikasi
Dashboard sebagian di-cache Memcached	Dashboard tidak di-cache; Memcached hanya untuk <code>product:<SKU></code> , izin role, aturan validasi file (Lampiran B).
Menu Sales: "Sales Orders, Customers, Reports"	Sidebar Sales: Dashboard, Products, Sales Orders, Reports. Customers tidak ada di sidebar (hanya via URL).
Menu Warehouse: "Purchase Orders, Sales Orders (untuk issue), Stock Ledger"	Tidak ada menu Sales Orders. Issue dilakukan lewat Goods Issue Queue di dashboard. Sidebar juga memuat Products dan Reports.
Admin menu: "Master Data, Procurement, Sales, Inventory, Reports, Users"	Lengkap: ditambah Dashboard; Master Data berisi Products, Categories, Warehouses, Suppliers, Customers; Users ada di grup Administration.
Tombol "Terima", "Issue"	Label di aplikasi: "Receive Goods", "Issue Goods" (juga "Process Issue"/"Receive" di antrean dashboard).
Kartu Stok "tidak bisa diubah"	Tidak ada rute/fitur edit atau hapus baris ledger. Skema tidak punya trigger DB yang melarangnya — jadi perlindungan ada di tingkat aplikasi.
Sesi "bila Redis mati login tetap jalan"	Benar, dengan batasan: sesi lama tidak ikut pindah (Lampiran A.2).

F.3 Catatan jujur untuk tanya-jawab

- Halaman daftar Categories, Warehouses, Suppliers, dan Products hanya butuh login; Sales & Warehouse yang mengetik URL-nya melihat data read-only (tanpa tombol kelola). Yang dijaga ketat di server adalah aksi tulis. `/customers` dijaga `customers.view` (Warehouse → 403).
- Menu disembunyikan lewat `PermissionService` (yang di-cache 1 jam); guard server memakai data izin yang sama. Perubahan izin di database dapat tertunda hingga TTL habis.

- Menandai **inferensi** = disimpulkan dari pembacaan kode, belum dijalankan end-to-end.